

Lập trình .NET (ASP.NET Core Web API)

**Buổi 3: Controller, Routing,
HTTP Methods, RESTful
API, Model Binding,
Action Result**



1. Lý thuyết

1.1. Controller

1.2. Routing

1.3. HTTP Methods

1.4. RESTful API

1.5. Action Result

1.6. Status Codes

2. Thực hành

Xây dựng CRUD với dữ liệu giả

1.1. Controller

Controller Trong ASP.NET Core

Lớp tiếp nhận và xử lý các **HTTP request** đến. Nhóm các endpoint (action) liên quan đến cùng một **resource**. Trong Web API: kế thừa **ControllerBase** (không phải Controller).

1.1. Controller (tt)

Controller Trong ASP.NET Core

	ControllerBase	Controller
Dùng cho		MVC (có View)
Trả về	Dữ liệu (JSON)	View + Dữ liệu
Hỗ trợ View	×	✓

[ApiController] – bắt các hành vi chuẩn cho Web API:

- Tự động trả “400 Bad Request” khi “ModelState” không hợp lệ
- Tự suy luận binding source (“[FromBody]”, “[FromRoute]”...)
- Yêu cầu bắt buộc dùng **Attribute Routing**

 **Ghi nhớ:** Web API dùng “ControllerBase”, không cần View. Luôn gắn “[ApiController]”.

Routing Trong ASP.NET Core

2 loại Routing:

1. Conventional Routing (ít dùng trong API):

```
app.MapControllerRoute(  
    name: "default",  
    pattern: "{controller}/{action}/{id?}");
```

2. Attribute Routing (chuẩn cho Web API)

```
[ApiController]  
[Route("api/[controller]")] // → /api/students  
public class StudentsController : ControllerBase  
{  
    [HttpGet]                // → GET /api/students  
    [HttpGet("{id}")]        // → GET /api/students/{id}  
    [HttpPost]               // → POST /api/students  
    [HttpPut("{id}")]        // → PUT /api/students/{id}  
    [HttpDelete("{id}")]     // → DELETE  
    /api/students/{id}  
}
```

✓ **Best Practice:** Luôn dùng Attribute Routing trong Web API. “[controller]” sẽ tự thay bằng tên controller (bỏ hậu tố “Controller”).

1.3. HTTP Methods

HTTP Methods & Ý Nghĩa

Method	Mục đích	Idempotent?
GET	Lấy dữ liệu	✓
POST	Tạo mới resource	✗
PUT	Cập nhật toàn bộ resource	✓
PATCH	Cập nhật một phần resource	✗
DELETE	Xóa resource	✓

Idempotent: gọi nhiều lần cho kết quả (trạng thái server) giống như gọi một lần.

1.3. HTTP Methods (tt)

Ví dụ với Student API

Method	URI	Mô tả
GET	/api/students	Lấy tất cả sinh viên
GET	/api/students/{id}	Lấy một sinh viên
POST	/api/students	Tạo sinh viên mới
PUT	/api/students/{id}	Cập nhật toàn bộ
DELETE	/api/students/{id}	Xóa sinh viên

1.4. RESTful API

RESTful API là gì?



1.4. RESTful API (tt)

RESTful API là gì?

REST – Representational State Transfer. Kiến trúc thiết kế API theo chuẩn HTTP. Mỗi **resource** (tài nguyên) được định danh bằng **URI**. Thao tác với resource qua **HTTP Methods**.

6 nguyên tắc REST

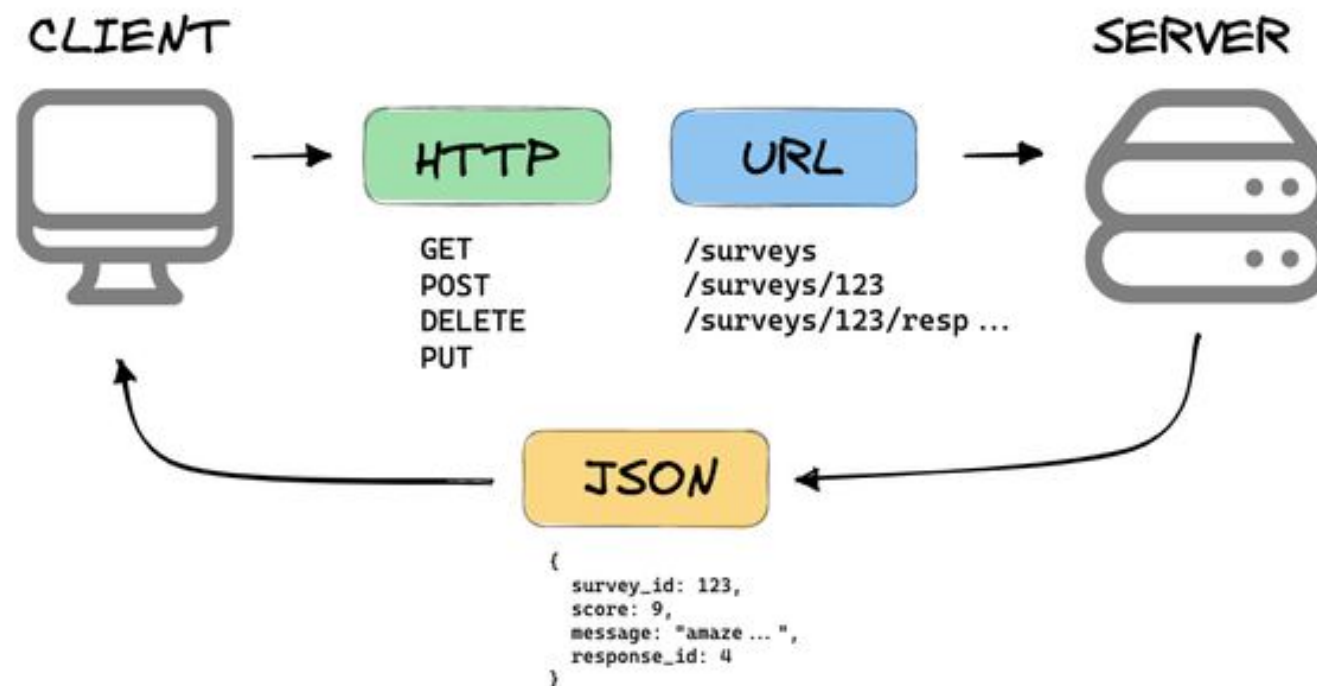
Nguyên tắc	Ý nghĩa
Stateless	Server không lưu trạng thái của client
Client-Server	Tách biệt giao diện và dữ liệu
Uniform Interface	Giao diện nhất quán qua URI + Method
Cacheable	Response có thể được cache
Layered System	Client không cần biết mình gọi trực tiếp server hay qua trung gian
Code on Demand	(Tùy chọn) Server gửi code về client

Ghi nhớ: REST không phải giao thức – là quy ước thiết kế API.

1.4. RESTful API (tt)

RESTful API là gì?

WHAT IS A REST API?



1.4. RESTful API (tt)

Model Binding – Nhận Dữ Liệu Từ Request

Dữ liệu có thể đến từ nhiều nguồn:

Attribute	Nguồn	Ví dụ
[FromRoute]	URL path	/api/students/5
[FromQuery]	Query string	?page=1&size=10
[FromBody]	Request body (JSON)	{ "name": "An" }
[FromHeader]	HTTP header	Authorization: Bearer ...

Ví dụ thực tế:

```
// GET /api/students/5?includeGrades=true  
[HttpGet("{id}")]  
public IActionResult Get(  
    [FromRoute] int id,  
    [FromQuery] bool includeGrades = false)  
{  
    // ...  
}
```

```
// POST /api/students  
[HttpPost]  
public IActionResult Create([FromBody]  
    CreateStudentDto dto)  
{  
    // ...  
}
```



Ghi nhớ: Với “[ApiController]”, ASP.NET Core **tự suy luận** nguồn binding nên nhiều trường hợp không cần ghi attribute.

IActionResult là interface cho mọi loại response:

Method	Status Code	Dùng khi
Ok(data)	200 OK	Thành công, có dữ liệu
CreatedAtAction(...)	201 Created	Tạo mới thành công
NoContent()	204 No Content	Thành công, không có dữ liệu
BadRequest(error)	400 Bad Request	Dữ liệu đầu vào sai
Unauthorized()	401 Unauthorized	Chưa xác thực
Forbid()	403 Forbidden	Không có quyền
NotFound()	404 Not Found	Không tìm thấy resource
StatusCode(500)	500 Internal Server Error	Lỗi server

⚠ Lỗi thường gặp: Trả Ok(null) khi không tìm thấy thay vì NotFound().

2.1. Xây dựng CRUD với dữ liệu giả

Định nghĩa “Student” Model:

```
// Models/Student.cs
```

```
namespace StudentApi.Models;
```

```
public class Student
```

```
{
```

```
    public int Id { get; set; }
```

```
    public string Name { get; set; } = string.Empty;
```

```
    public string Major { get; set; } = string.Empty;
```

```
    public double GPA { get; set; }
```

```
}
```

2.1. Xây dựng CRUD với dữ liệu giả

Định nghĩa “Student” Model:

```
// Models/Student.cs
```

```
namespace StudentApi.Models;
```

```
public class Student
```

```
{
```

```
    public int Id { get; set; }
```

```
    public string Name { get; set; } = string.Empty;
```

```
    public string Major { get; set; } = string.Empty;
```

```
    public double GPA { get; set; }
```

```
}
```

2.1. Xây dựng CRUD với dữ liệu giả (tt)

Dữ liệu giả (in-memory) cho demo:

```
private static readonly List<Student> _students = new()  
{  
    new Student { Id = 1, Name = "An Nguyễn", Major = "CNTT", GPA = 3.5 },  
    new Student { Id = 2, Name = "Bình Trần", Major = "CNTT", GPA = 2.8 },  
    new Student { Id = 3, Name = "Chi Lê", Major = "HTTT", GPA = 3.9 },  
};
```

2.1. Xây dựng CRUD với dữ liệu giả (tt)

GET: Lấy dữ liệu

[ApiController]

[Route("api/[controller]")]

public class StudentsController : ControllerBase

```
{  
    private static readonly List<Student> _students = new() { /* ... */ };  
  
    // GET /api/students  
  
    [HttpGet]  
    public IActionResult GetAll()  
    {  
        return Ok(_students);  
    }  
}
```

2.1. Xây dựng CRUD với dữ liệu giả (tt)

GET: Lấy dữ liệu

```
// GET /api/students/1
```

```
[HttpGet("{id:int}")]
```

```
public IActionResult GetById(int id)
```

```
{
```

```
    var student = _students.FirstOrDefault(s => s.Id == id);
```

```
    if (student is null)
```

```
        return NotFound(new { message = $"Không tìm thấy sinh viên Id = {id}" });
```

```
    return Ok(student);
```

```
}
```

```
}
```

2.1. Xây dựng CRUD với dữ liệu giả (tt)

POST: Tạo Mới Resource

// POST /api/students

[HttpPost]

public IActionResult Create([FromBody] Student student)

{

if (student is null)

return BadRequest(new { message = "Dữ liệu không hợp lệ." });

// Gán Id mới

student.Id = _students.Any()

? _students.Max(s => s.Id) + 1

: 1;

_students.Add(student);

// Trả 201 Created với Location header

return CreatedAtAction(

nameof(GetById),

new { id = student.Id },

student);

}

 **Ghi nhớ:** “POST” trả về “201 Created” kèm URI của resource vừa tạo, không phải “200 OK”.

2.1. Xây dựng CRUD với dữ liệu giả (tt)

PUT: Cập Nhật Resource

```
// PUT /api/students/1
```

```
[HttpPut("{id:int}")]
```

```
public IActionResult Update(int id, [FromBody] Student updated)
```

```
{
```

```
    var student = _students.FirstOrDefault(s => s.Id == id);
```

```
    if (student is null)
```

```
        return NotFound(new { message = $"Không tìm thấy sinh viên Id = {id}" });
```

```
    // Cập nhật toàn bộ thuộc tính
```

```
    student.Name = updated.Name;
```

```
    student.Major = updated.Major;
```

```
    student.GPA = updated.GPA;
```

```
    return NoContent(); // 204 - Thành công nhưng không trả dữ liệu
```

```
}
```

2.1. Xây dựng CRUD với dữ liệu giả (tt)

PUT: Cập Nhật Resource

```
// PUT /api/students/1
```

```
[HttpPut("{id:int}")]
```

```
public IActionResult Update(int id, [FromBody] Student updated)
```

```
{  
    var student = _students.FirstOrDefault(s => s.Id == id);  
  
    if (student is null)  
        return NotFound(new { message = $"Không tìm thấy sinh viên Id = {id}" });  
  
    // Cập nhật toàn bộ thuộc tính  
    student.Name = updated.Name;  
    student.Major = updated.Major;  
    student.GPA = updated.GPA;  
  
    return NoContent(); // 204 - Thành công nhưng không trả dữ liệu  
}
```



Lỗi thường gặp: Dùng PUT để cập nhật một phần → nên dùng

PATCH.

2.1. Xây dựng CRUD với dữ liệu giả (tt)

DELETE: Xóa Resource

```
// DELETE /api/students/1
```

```
[HttpDelete("{id:int}")]
```

```
public IActionResult Delete(int id)
```

```
{
```

```
    var student = _students.FirstOrDefault(s => s.Id == id);
```

```
    if (student is null)
```

```
        return NotFound(new { message = $"Không tìm thấy sinh viên Id = {id}" });
```

```
    _students.Remove(student);
```

```
    return NoContent(); // 204 - Xóa thành công
```

```
}
```

2.1. Xây dựng CRUD với dữ liệu giả (tt)

Tổng kết CRUD endpoints:

Operation	Method	URI	Status Code
Read All	GET	/api/students	200
Read One	GET	/api/students/{id}	200 / 404
Create	POST	/api/students	201
Update	PUT	/api/students/{id}	204 / 404
Delete	DELETE	/api/students/{id}	204 / 404

2.1. Xây dựng CRUD với dữ liệu giả (tt)

Route Constraints & Naming Convention

[HttpGet("{id:int}")] // id phải là int

[HttpGet("{name:alpha}")] // name chỉ chứa chữ cái

[HttpGet("{id:min(1)}")] // id ≥ 1

✓ Đúng	✗ Sai
/api/students	/api/getStudents
/api/students/{id}	/api/student/getById?id=1
/api/students/{id}/grades	/api/getStudentGrades

Quy tắc:

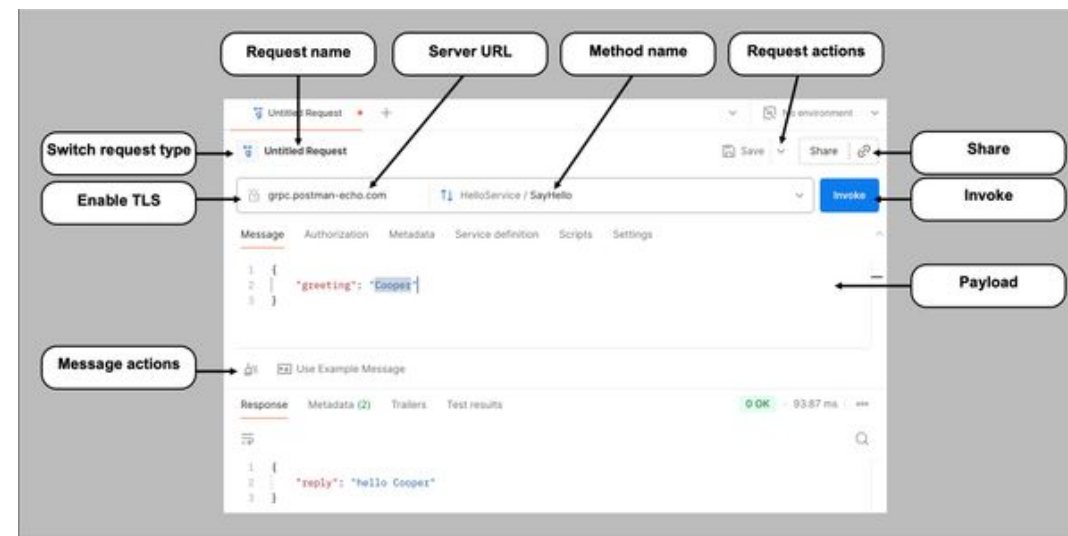
- Dùng **danh từ số nhiều** (students, products, orders)
- Dùng **lowercase** và **kebab-case** nếu nhiều từ
- Không đặt **verb** (get, create, delete) trong URI

✓ **Best Practice:** URI định danh **resource**, HTTP Method xác định **hành động**.

2.1. Xây dựng CRUD với dữ liệu giả (tt)

Demo: CRUD Hoàn Chỉnh với Dữ Liệu Giả

1. Tạo project “StudentApi”
2. Tạo “Models/Student.cs”
3. Tạo “Controllers/StudentsController.cs” với:
 - “GET /api/students” - GetAll
 - “GET /api/students/{id}” - GetById
 - “POST /api/students” - Create
 - “PUT /api/students/{id}” - Update
 - “DELETE /api/students/{id}” - Delete
4. Test toàn bộ CRUD trên Swagger
5. Test với Postman (tạo Collection “StudentApi”)



2.3. Bài thực hành

Yêu cầu: Xây dựng CRUD API với dữ liệu in-memory.

1. Tạo “ProductsController” với 2 endpoint:

- “GET /api/products” → trả danh sách 3 sản phẩm
- “GET /api/products/{id}” → trả 1 sản phẩm hoặc 404

2. Thêm endpoint “POST /api/products” để tạo sản phẩm mới.

- Trả về “201 Created” kèm dữ liệu sản phẩm vừa tạo.
- Test bằng Swagger và Postman.

3. Thêm “PUT /api/products/{id}” và “DELETE /api/products/{id}”.

- Đảm bảo trả đúng status code: 204, 404.
- Xử lý trường hợp `id` không tồn tại.

2.3. Bài thực hành (tt)

Yêu cầu: Xây dựng CRUD API với dữ liệu in-memory.

4. Thêm “GET /api/products?category=Electronics&minPrice=100”.

- Dùng “[FromQuery]” để lọc sản phẩm theo “category” và “minPrice”.
- Dùng LINQ để thực hiện lọc.

5. Thiết kế URI chuẩn REST cho hệ thống quản lý đơn hàng:

- Đơn hàng (“Order”), chi tiết đơn hàng (“OrderItem”)
- Mỗi đơn hàng có nhiều “OrderItem”
- Thiết kế 8 endpoints: CRUD cho Order + CRUD cho OrderItem (nested route)



Thank You!